

Kite Ledger Mobile Application – Standard Operating Procedure (SOP)

1. Document Information

Item	Details
Project Name	Kite Ledger
Application Type	Mobile Expense Tracking Application
Frontend	React Native
Development Framework	Expo
Programming Language	JavaScript / TypeScript
Backend	JavaScript / TypeScript
Data Storage	AsyncStorage
Version Control	Git
Repository	GitHub
Supported Platform	Android / iOS

2. Purpose

The purpose of this Standard Operating Procedure (SOP) is to document the development, configuration, testing, and maintenance procedures for the **Kite Ledger Mobile Application**.

Kite Ledger is a mobile expense management application designed to help users record and monitor their daily and monthly expenses.

The application allows users to:

- Add expense entries.
- View expense records.
- Edit existing expenses.
- Delete expenses.
- View daily expenses.
- View monthly expenses.
- Organize expenses by category.
- Store application data locally on the device.

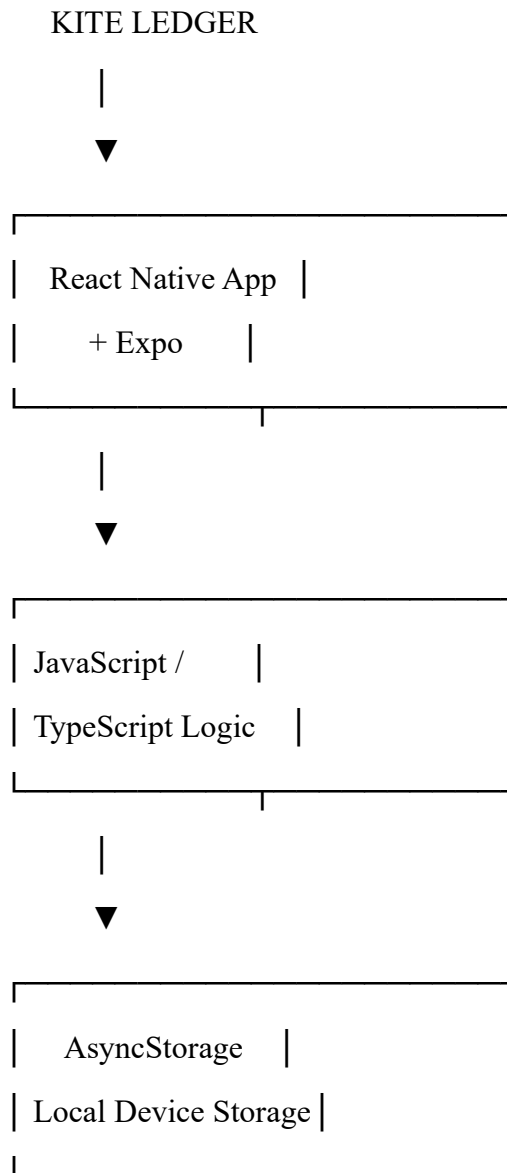
3. Application Overview

Kite Ledger is developed as a mobile application using React Native and Expo.

Application data is stored locally using **AsyncStorage**.

The application does not currently use a cloud database or external authentication service.

High-Level Architecture



4. Technology Stack

4.1 React Native

React Native is used to develop the mobile application's user interface and functionality.

It allows the application to be developed using JavaScript/TypeScript while targeting mobile platforms.

4.2 Expo

Expo is used as the development framework and toolchain for the React Native application.

Expo provides tools for:

- Application development.
- Local development.
- Running the application on physical devices.
- Running the application on emulators.
- Application configuration.
- Building application releases.

4.3 JavaScript / TypeScript

JavaScript and TypeScript are used for application development.

They are used for:

- Application logic.
- UI components.
- Navigation.
- Expense calculations.
- Data handling.
- Local storage operations.
- Backend-related logic, where applicable.

4.4 AsyncStorage

AsyncStorage is used as the application's local storage mechanism.

It is a persistent key-value storage system available within the mobile application.

It can be used to store:

- Expense entries.
- User preferences.
- Application settings.
- Other non-sensitive application data.

Important

AsyncStorage is **local storage**, not a centralized cloud database.

The stored data exists on the user's device and is not automatically synchronized between multiple devices.

5. Prerequisites

The following software should be installed before starting development:

1. Node.js
2. npm
3. Visual Studio Code
4. Git
5. Expo development tools
6. Android Studio (optional, for Android Emulator)
7. Expo Go (for physical-device testing, if applicable)

6. Verify Development Environment

Open Command Prompt or PowerShell.

Check Node.js:

```
node -v
```

Check npm:

```
npm -v
```

Check npx:

```
npx -v
```

If all commands return valid versions, the basic Node.js environment is available.

7. Project Setup

Create the Expo project using:

```
npx create-expo-app@latest kite-ledger-mobile
```

Navigate to the project:

```
cd kite-ledger-mobile
```

Install dependencies:

```
npm install
```

Start the Expo development server:

```
npx expo start
```

8. Project Structure

A recommended structure for the Kite Ledger application is:

kite-ledger-mobile/

```
|
|
├── app/
|   ├── index.tsx
|   ├── login.tsx
|   ├── signup.tsx
|   ├── home.tsx
|   ├── add-entry.tsx
|   ├── manage-entries.tsx
|   ├── daily-calendar.tsx
|   └── last-month.tsx
├── components/
|   ├── Button.tsx
|   ├── Input.tsx
|   └── ExpenseCard.tsx
├── services/
|   └── storage.ts
|
├── assets/
├── package.json
├── app.json
├── tsconfig.json
├── .gitignore
└── README.md
```

The actual project structure may differ depending on the implementation and Expo Router configuration.

9. Application Screens

The Kite Ledger application consists of screens for managing expenses.

9.1 Login / Entry Screen

If a login screen is implemented, it is used to provide access to the application.

The screen may contain:

- User information.
- Login fields.
- Navigation to the main application.

No external authentication service is currently required by the application.

10. Home Screen

The Home screen provides an overview of the user's expense information.

It can display:

- Today's total expense.
- Current month's total expense.
- Recent expense entries.
- Expense categories.
- Navigation to other sections.

Example:

KITE LEDGER

Today's Expense

₹1,250

Monthly Expense

₹18,450

Recent Entries

Food	₹250
------	------

Transport	₹120
-----------	------

Shopping	₹850
----------	------

+ Add Expense

11. Add Entry

The Add Entry screen allows users to create a new expense record.

Typical fields include:

- Amount.
- Category.
- Description.
- Date.

Example:

Amount: ₹500

Category: Food

Description: Lunch

Date: 31-08-2026

Validation

The application should validate:

- Amount should be greater than zero.
- Required fields should not be empty.
- Date should be valid.
- Invalid values should generate an appropriate error message.

12. Manage Entries

The Manage Entries screen allows users to manage stored expenses.

Functions include:

- View entries.
- Edit entries.
- Delete entries.
- Search entries, if implemented.
- Filter entries, if implemented.

Example:

Expense List

Food	₹250
Transport	₹120
Shopping	₹850

[Edit] [Delete]

13. Daily Calendar

The Daily Calendar allows users to select a specific date and view expenses recorded for that date.

Process:

Select Date



Read AsyncStorage



Filter Entries



Display Daily Expenses

The application should display only the expenses corresponding to the selected date.

14. Monthly Expense View

The monthly expense screen provides an overview of expenses for a selected month.

The application can calculate the monthly total by adding all expense entries belonging to the selected month.

Example:

January 2026

Food	₹4,500
------	--------

Transport	₹2,000
-----------	--------

Shopping	₹3,500
----------	--------

Total	₹10,000
-------	---------

15. AsyncStorage Data Management

AsyncStorage is used to persist application data locally.

The general process is:

User Adds Expense



Validate Data



Convert Data to Storage Format



AsyncStorage



Data Saved Locally

When retrieving data:

Application Opens



Read AsyncStorage



Retrieve Stored Data



Convert Data



Display Expenses

16. Expense Data Example

Expense data may be represented in JSON format.

Example:

```
{  
  "id": "001",  
  "amount": 500,  
  "category": "Food",  
  "description": "Lunch",  
  "date": "2026-08-31"  
}
```

Multiple entries may be stored as an array:

```
[  
  {  
    "id": "001",  
    "amount": 500,  
    "category": "Food",  
    "description": "Lunch",  
    "date": "2026-08-31"  
  },  
  {  
    "id": "002",  
    "amount": 120,  
    "category": "Transport",  
    "description": "Bus",  
    "date": "2026-08-31"  
  }  
]
```

The exact data structure should follow the implementation in the application.

17. Data Operations

The application should support the following basic operations.

Create

Add a new expense:

User Input



Validation



Create Expense Object



Save to AsyncStorage

Read

Retrieve existing expenses:

AsyncStorage



Read Stored Data



Parse Data



Display Entries

Update

Edit an existing expense:

Select Expense



Modify Data



Update Array



Save to AsyncStorage

Delete

Delete an expense:

Select Expense



Confirm Delete



Remove Expense



Save Updated Data

18. Application Testing

Testing should be performed after implementing each major feature.

18.1 Functional Testing

Verify:

- Application launches successfully.
- Home screen loads.
- Expense can be added.
- Expense is saved.
- Expense can be viewed.
- Expense can be edited.
- Expense can be deleted.
- Daily expenses are displayed correctly.
- Monthly expenses are calculated correctly.
- Data remains available after restarting the application.

19. AsyncStorage Testing

Verify that:

1. New data is successfully stored.
2. Stored data can be retrieved.
3. Data can be updated.
4. Data can be deleted.
5. Application restart does not unintentionally remove stored data.
6. Invalid or corrupted stored data is handled safely.

20. Mobile Device Testing

The application should be tested on:

- Android Emulator.
- Physical Android device.
- iOS Simulator, if available.
- Physical iOS device, if applicable.

For physical-device testing:

`npx expo start`

Then connect the device using the available Expo development workflow.

21. Backend

The current application does not use a separate cloud backend service.

Any JavaScript/TypeScript backend-related code included in the project should be documented according to its actual implementation.

Do not introduce or configure additional backend technologies unless they are added to the project.

The current data storage mechanism is:

React Native



Application Logic



AsyncStorage



Local Device

22. Security

Since AsyncStorage is local application storage, sensitive information should not be stored in plain text.

The following information should not be stored directly in AsyncStorage unless properly protected:

- Passwords.
- Authentication secrets.
- Private keys.
- API secrets.
- Other highly sensitive credentials.

The application should also avoid exposing sensitive information through:

- Console logs.
- Error messages.
- Source code.
- Git commits.

23. Git Configuration

Initialize Git in the project:

```
git init
```

Check the project status:

```
git status
```

Add project files:

```
git add .
```

Create a commit:

```
git commit -m "Initial Kite Ledger mobile application"
```

24. GitHub Repository

Create a GitHub repository for the project.

Recommended repository name:

```
kite-ledger-mobile
```

Connect the local repository to GitHub:

```
git remote add origin <YOUR_GITHUB_REPOSITORY_URL>
```

Rename the branch:

```
git branch -M main
```

Push the project:

```
git push -u origin main
```

25. Git Development Workflow

For future changes:

```
git status
```

```
git add .
```

```
git commit -m "Describe the changes"
```

```
git push
```

Example:

```
git add .
```

```
git commit -m "Add monthly expense view"
```

```
git push
```

Recommended commit messages:

Add expense entry screen

Update home screen

Fix expense calculation

Add AsyncStorage integration

Fix delete expense functionality

Update daily calendar

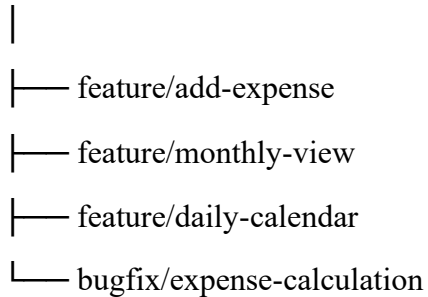
Improve UI validation

26. Recommended Branching

For larger changes, use feature branches.

Example:

main



Create a branch:

```
git checkout -b feature/add-expense
```

After testing the feature, merge it into the main branch.

27. .gitignore

The following files and folders should generally not be committed:

node_modules/

.expo/

dist/

build/

.env

.env.*

*.log

.DS_Store

The .gitignore file should be maintained according to the actual project requirements.

28. Application Startup Procedure

To start the application during development:

Step 1 – Open the project

Open the Kite Ledger project in Visual Studio Code.

Step 2 – Open Terminal

Navigate to the project directory.

Step 3 – Install dependencies

```
npm install
```

Step 4 – Start Expo

```
npx expo start
```

Step 5 – Run the application

Use the available Expo development options to run the application on:

- Android.
- iOS.
- Web, if supported by the project.

29. Troubleshooting

Application Does Not Start

Run:

```
npm install
```

Then:

```
npx expo start
```

If required, clear the Expo cache:

```
npx expo start -c
```

AsyncStorage Data Not Displaying

Check:

- Storage key.
- JSON parsing.
- JSON stringification.
- Data structure.
- AsyncStorage read operation.
- AsyncStorage write operation.

Typical flow:

JSON.stringify()

↓

AsyncStorage.setItem()

When reading:

AsyncStorage.getItem()

↓

JSON.parse()

Application Cannot Connect During Testing

If the project contains a separate local JavaScript/TypeScript service, verify:

- Service is running.
- Correct IP address is configured.
- Mobile device and computer are on the same network.
- Required port is accessible.
- Firewall is not blocking the connection.

If the application is entirely local, no network connection is required for expense storage.

30. Backup Considerations

Because expense information is stored using AsyncStorage, the application data is primarily associated with the device.

Therefore:

Device Storage



AsyncStorage



Local Application Data

If the application is removed or device data is cleared, locally stored data may be lost.

A future version of Kite Ledger could introduce cloud synchronization or backup if required.

31. Deployment / Release

Before creating a production release:

Application

- Test all screens.
- Test expense creation.
- Test expense editing.
- Test expense deletion.
- Test daily calculations.
- Test monthly calculations.
- Verify AsyncStorage persistence.
- Test on a physical device.

Code

- Remove unnecessary console logs.
- Check for errors and warnings.
- Update application version.
- Commit final changes.
- Push changes to GitHub.

Release

- Create production build.
- Install the release build on a test device.
- Perform final testing.
- Release the application.

32. Maintenance

Regular maintenance should include:

- Updating dependencies.
- Fixing application bugs.
- Improving UI/UX.
- Testing AsyncStorage compatibility.
- Reviewing application performance.
- Reviewing GitHub repository changes.
- Updating documentation.

Before performing major dependency updates, create a separate Git branch and test the application.

33. Project Documentation Structure

The recommended GitHub repository structure is:

kite-ledger-mobile/

```
|
|
|— app/
|   |— index.tsx
|   |— home.tsx
|   |— add-entry.tsx
|   |— manage-entries.tsx
|   |— daily-calendar.tsx
|   └— last-month.tsx
|
|— components/
|
|— services/
|
|— assets/
|
|— docs/
|   └— SOP.md
|
|— package.json
|— app.json
|— tsconfig.json
|— .gitignore
└— README.md
```

34. Recommended SOP Location

Store this document inside the GitHub repository:

docs/SOP.md

This keeps the operational documentation separate from the application source code.

35. Change Management

Whenever a major application change is made, update this SOP.

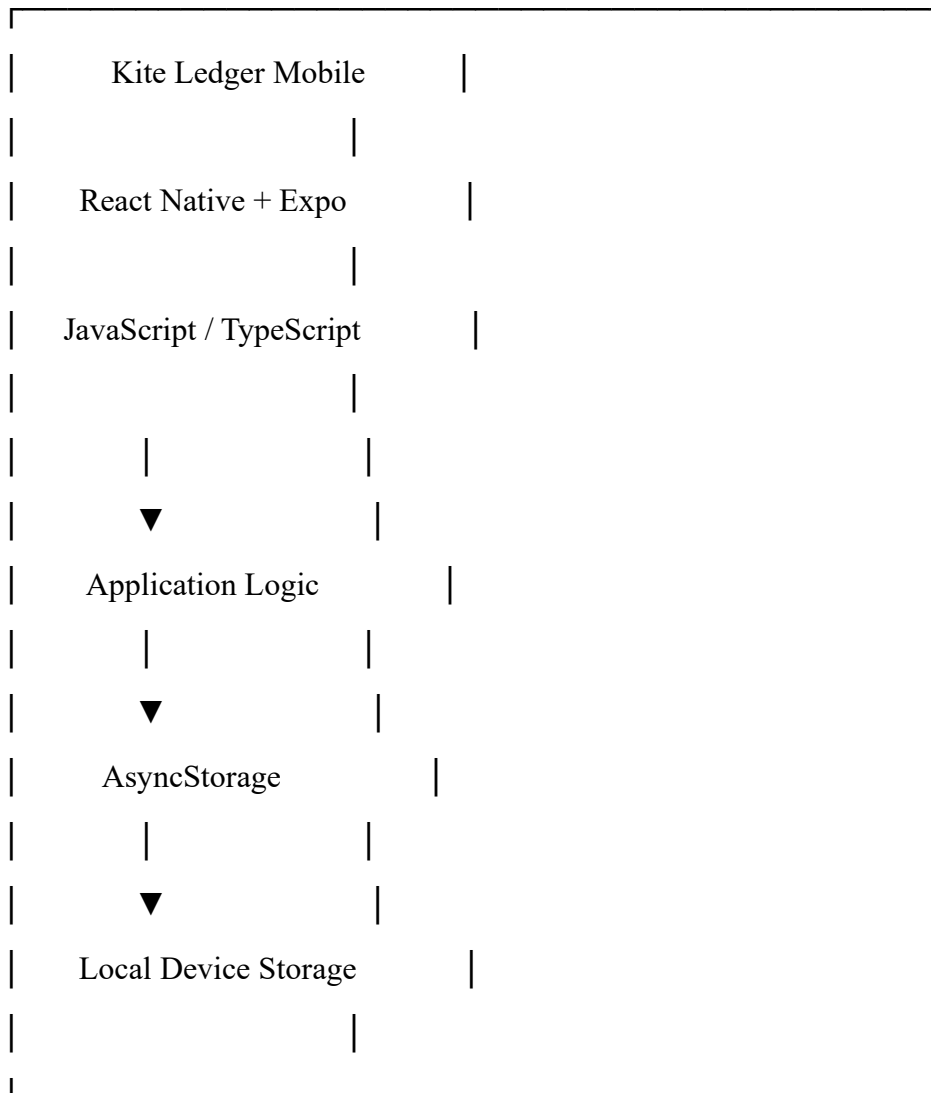
Examples:

- New technology added.
- Storage mechanism changed.
- New screen added.
- Authentication added.
- Backend introduced.
- Cloud database introduced.
- Deployment process changed.

The SOP should always represent the current architecture of the Kite Ledger application.

36. Current Architecture Summary

The current Kite Ledger architecture is:



Technology Summary

Frontend : React Native

Framework : Expo

Language : JavaScript / TypeScript

Storage : AsyncStorage

Authentication : Not currently used

Cloud Database : Not currently used

Cloud Backend : Not currently used

Version Control : Git / GitHub

37. Conclusion

Kite Ledger is a mobile expense tracking application developed using **React Native and Expo**, with application logic written in **JavaScript/TypeScript**.

Expense information and other application data are stored locally using **AsyncStorage**.

This SOP provides the standard procedure for:

- Setting up the development environment.
- Running the application.
- Understanding the application architecture.
- Managing expenses.
- Working with AsyncStorage.
- Testing the application.
- Managing the project using Git.
- Maintaining the GitHub repository.
- Preparing the application for release.

This document should be updated whenever the application's architecture, technology stack, storage mechanism, or deployment process changes.
